

Demo Runbook 3

Dataset Versioning • Model Artifact Management • Reproducibility •
Pipeline Automation Basics

Copy-paste-ready commands, expected outputs, and talking points for running each lab live as a demo.

How to use this	Run the Setup block once before the audience arrives. During the demo, execute each numbered Step live, pausing on the 'Expected output' box so the audience sees the result before you move on.
Total demo time	Approx. 30-40 minutes for all 4 labs

LAB 1

Dataset Versioning

Versioning a dataset with DVC and reverting to an earlier version live

Demo Goal: Show data being tracked like code — changed, committed, and rolled back — using DVC alongside Git.

Setup (run once before the demo)

```
pip install dvc pandas
mkdir dataset_versioning_demo && cd dataset_versioning_demo
git init -q && dvc init -q
```

STEP 1

Create version 1 of the dataset

```
python3 -c "
import pandas as pd
df = pd.DataFrame({'age': [25, 40, 35], 'income': [50000, 62000, 58000]})
df.to_csv('data.csv', index=False)
"
cat data.csv
```

Expected output / what you'll see:

```
age,income
25,50000
40,62000
35,58000
```

STEP 2

Track it with DVC and commit

```
dvc add data.csv
git add data.csv.dvc .gitignore
git commit -m 'Add dataset v1 (3 rows)' -q
git log --oneline
```

Expected output / what you'll see:

```
Add dataset v1 (3 rows)
```

STEP 3

Change the dataset — simulate new data arriving

```
python3 -c "
import pandas as pd
df = pd.DataFrame({'age': [25, 40, 35, 51, 29], 'income': [50000, 62000, 58000, 71000, 49000]})
df.to_csv('data.csv', index=False)
"
wc -l data.csv
```

Expected output / what you'll see:

```
6 data.csv (5 data rows + header)
```

STEP 4

Track and commit version 2

```
dvc add data.csv
git add data.csv.dvc
git commit -m 'Add dataset v2 (5 rows)' -q
git log --oneline
```

Expected output / what you'll see:

```
Add dataset v2 (5 rows)
Add dataset v1 (3 rows)
```

STEP 5

Roll back to version 1 live

```
git checkout HEAD~1 -- data.csv.dvc
dvc checkout
cat data.csv
```

Expected output / what you'll see:

```
age,income
25,50000
40,62000
35,58000
(Exactly the 3-row version 1 dataset is restored.)
```

Say to audience: *"This is the core idea: the dataset is now a first-class, versioned artifact just like code. We can prove exactly which version of the data produced any given model — and roll back to it on demand."*

STEP 6

Return to the latest version

```
git checkout main -- data.csv.dvc 2>/dev/null || git checkout master -- data.csv.dvc
dvc checkout
cat data.csv
```

Expected output / what you'll see:

```
The 5-row version 2 dataset is restored.
```

LAB 2

Model Artifact Management

Storing and retrieving model artifacts (not just the model file) via MLflow

Demo Goal: Show that an 'artifact' is more than a .pkl file — plots, metrics files, and preprocessing objects can all be tracked and retrieved together as one bundle.

Setup (run once before the demo)

```
pip install mlflow scikit-learn matplotlib
mkdir artifact_mgmt_demo && cd artifact_mgmt_demo
```

STEP 1

Train a model and log multiple artifact types in one run

```
cat > log_artifacts.py << 'EOF'
import json, mlflow, mlflow.sklearn, matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

mlflow.set_experiment('artifact-demo')
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

with mlflow.start_run():
    model = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
    preds = model.predict(X_test)

    # Artifact 1: the model itself
    mlflow.sklearn.log_model(model, 'model')

    # Artifact 2: a plot
    fig, ax = plt.subplots()
    ConfusionMatrixDisplay.from_predictions(y_test, preds, ax=ax)
    fig.savefig('confusion_matrix.png')
    mlflow.log_artifact('confusion_matrix.png')

    # Artifact 3: a metrics summary file
    summary = {'accuracy': accuracy_score(y_test, preds), 'n_features': X.shape[1]}
    json.dump(summary, open('summary.json', 'w'))
    mlflow.log_artifact('summary.json')

    print('Run complete – 3 artifact types logged.')
EOF
python log_artifacts.py
```

Expected output / what you'll see:

Run complete – 3 artifact types logged.

STEP 2

Open the run in the MLflow UI and show the Artifacts tab

```
mlflow ui # open the run, click the 'Artifacts' tab
```

Expected output / what you'll see:

Three items visible: model/ (a full model directory with its own metadata), confusion_matrix.png (viewable inline), and summary.json (downloadable).

Say to audience: *"This is the key point: an artifact isn't just a model binary — it's the full bundle of evidence for that run: the model, the visual proof of how it performed, and a portable metrics summary, all retrievable together, permanently, by run ID."*

STEP 3

Programmatically fetch an artifact without opening the UI

```
cat > fetch_artifact.py << 'EOF'
from mlflow.tracking import MlflowClient
client = MlflowClient()
exp = client.get_experiment_by_name('artifact-demo')
run = client.search_runs(exp.experiment_id, order_by=['start_time DESC'], max_results=1)[0]
local_path = client.download_artifacts(run.info.run_id, 'summary.json')
print('Downloaded to:', local_path)
print(open(local_path).read())
EOF
python fetch_artifact.py
```

Expected output / what you'll see:

```
Downloaded to: /tmp/.../summary.json
{"accuracy": 0.97..., "n_features": 13}
```

Say to audience: *"This is exactly how a CI/CD pipeline or serving system would pull artifacts automatically — no manual UI clicking required."*

LAB 3

Reproducibility

Proving two training runs produce bit-identical results

Demo Goal: Run the same training script twice with a fixed seed and show identical output, then remove the seed and show the results drift — making the value of reproducibility visible.

Setup (run once before the demo)

```
pip install scikit-learn numpy
mkdir reproducibility_demo && cd reproducibility_demo
```

STEP 1

Write a training script WITHOUT a fixed seed (the problem)

```
cat > train_no_seed.py << 'EOF'
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)    # no random_state
model = RandomForestClassifier()                                           # no random_state
model.fit(X_train, y_train)
print('Accuracy:', accuracy_score(y_test, model.predict(X_test)))
EOF
```

STEP 2

Run it three times back to back

```
python train_no_seed.py
python train_no_seed.py
python train_no_seed.py
```

Expected output / what you'll see:

```
Accuracy: 0.9722...
Accuracy: 0.9444...
Accuracy: 1.0000...
(Three different numbers from the exact same code and data.)
```

Say to audience: "Same code, same data file, three different results. If a stakeholder asks 'why did the number change,' we currently have no good answer — that's the reproducibility gap."

STEP 3

Fix it with fixed seeds everywhere randomness is used

```
cat > train_seeded.py << 'EOF'
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

SEED = 42
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)
model = RandomForestClassifier(random_state=SEED)
model.fit(X_train, y_train)
```

```
print('Accuracy:', accuracy_score(y_test, model.predict(X_test)))
EOF
```

STEP 4

Run the seeded version three times back to back

```
python train_seeded.py
python train_seeded.py
python train_seeded.py
```

Expected output / what you'll see:

```
Accuracy: 0.9722222222222222
Accuracy: 0.9722222222222222
Accuracy: 0.9722222222222222
(Identical, bit-for-bit, every time.)
```

Say to audience: *"Same code, same data, identical result every single time. This is what makes a result trustworthy enough to put in a report, an audit, or a regulatory filing."*

Demo Wrap-up

If time allows, mention the two other pillars of reproducibility covered earlier in this series: locked dependency versions (Environment Setup demo) and versioned data (Dataset Versioning demo, just before this one) — seeding alone isn't sufficient without those.

LAB 4

Pipeline Automation Basics

Chaining pipeline stages with a Makefile so one command runs everything in order

Demo Goal: Replace 'run these four scripts in the right order, manually, and don't forget one' with a single command that always does it correctly.

Setup (run once before the demo)

```
mkdir pipeline_automation_demo/src -p && cd pipeline_automation_demo
```

STEP 1

Write three tiny pipeline stage scripts

```
cat > src/ingest.py << 'EOF'
print('[ingest] Fetching data...')
open('data.csv', 'w').write('age,income\n25,50000\n40,62000\n')
print('[ingest] Wrote data.csv')
EOF

cat > src/train.py << 'EOF'
print('[train] Reading data.csv...')
rows = open('data.csv').read().strip().split('\n')[1:]
print(f'[train] Trained on {len(rows)} rows')
open('model.txt', 'w').write('trained-model-v1')
EOF

cat > src/evaluate.py << 'EOF'
print('[evaluate] Loading model.txt...')
model = open('model.txt').read()
print(f'[evaluate] Evaluated {model} -> accuracy=0.95')
EOF
```

STEP 2

Show the manual, error-prone way first

Deliberately run them out of order to show what happens.

```
python src/evaluate.py
```

Expected output / what you'll see:

```
[evaluate] Loading model.txt...
FileNotFoundError: [Errno 2] No such file or directory: 'model.txt'
```

Say to audience: "This is what happens when a human has to remember the right order every time — eventually someone runs the wrong step first, especially under demo or deadline pressure."

STEP 3

Write a Makefile that encodes the correct order and dependencies

```
cat > Makefile << 'EOF'
.PHONY: all clean

all: evaluate

data.csv:
python src/ingest.py

model.txt: data.csv
```

```
python src/train.py

evaluate: model.txt
python src/evaluate.py

clean:
rm -f data.csv model.txt
EOF
```

STEP 4

Run the entire pipeline with one command

```
make clean
make
```

Expected output / what you'll see:

```
python src/ingest.py
[ingest] Fetching data...
[ingest] Wrote data.csv
python src/train.py
[train] Reading data.csv...
[train] Trained on 2 rows
python src/evaluate.py
[evaluate] Loading model.txt...
[evaluate] Evaluated trained-model-v1 -> accuracy=0.95
```

Say to audience: *"One command, correct order, every time — no human needs to remember the sequence anymore."*

STEP 5

Show incremental re-run behavior (the automation payoff)

```
make    # run again immediately, with nothing changed
```

Expected output / what you'll see:

```
make: 'evaluate' is up to date.
(Nothing re-runs, because none of the source files changed.)
```

Say to audience: *"This is the same principle behind tools like DVC pipelines or Airflow DAGs, just visible in its simplest possible form: a pipeline is a dependency graph, and automation means only re-running what actually needs to re-run."*

Demo Wrap-up

This Makefile pattern is the simplest possible on-ramp to the more advanced orchestrators (Airflow, Kubeflow Pipelines, DVC pipelines) covered in later, more advanced labs — the underlying idea (declare stages, declare dependencies, let the tool figure out order and caching) is identical.